

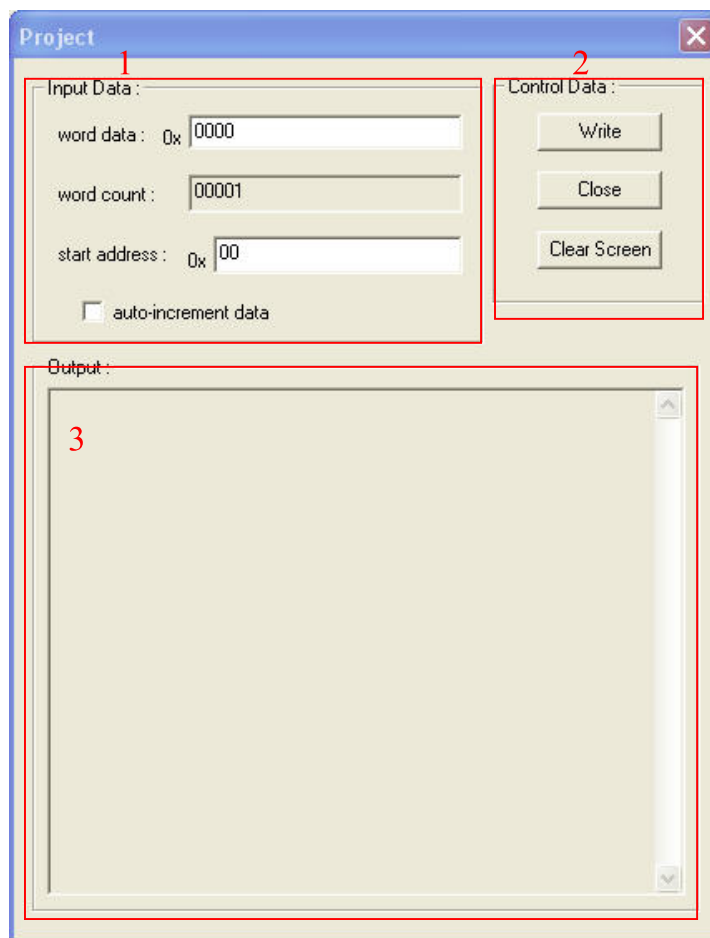
บทที่ 4

การทดลองและผลการทดลอง

ในบทนี้จะกล่าวถึงการทดลองในการเขียน driver ทั้ง 2 แบบ คือ user-mode driver และ kernel-mode driver รวมถึงการดีบั๊กโปรแกรม kernel-mode driver ด้วยโปรแกรม debugprint

4.1 การทดลองการเขียน User-mode driver

การเขียนโปรแกรม user-mode driver จะใช้ Visual C++ 6.0 ในการเขียนและ Compile ซึ่งมีวิธีการเขียนเหมือนกับการเขียนโปรแกรมทั่วไป เพียงแต่จะต้องเรียกใช้ฟังก์ชัน Win32 API หน้าตาของโปรแกรมดังแสดงในรูปที่ 4.1



รูปที่ 4.1 User interface ของโปรแกรม

1. ส่วนที่ 1

1. Edit box “word data”: ใส่ข้อมูลที่ต้องการจะเขียนไปยัง board
2. Edit box “word count”: ใส่จำนวนครั้งที่ต้องการจะส่งข้อมูลออกไป
3. Edit box “start address”: ใส่ address เริ่มต้นของการส่งข้อมูล
4. Radio box “auto-increment address”: เลือกถ้าผู้ใช้ต้องการให้เพิ่ม address อัตโนมัติเมื่อทำการอ่านเขียน

2. ส่วนที่ 2

1. Button “Write”: คลิกเพื่อเขียนข้อมูลไปยัง board
2. Button “Close”: คลิกเพื่อปิดโปรแกรม
3. Button “Clear Screen”: คลิกเพื่อลบข้อมูลทั้งหมดใน edit box

3. ส่วนที่ 3

1. Edit box ที่ใหญ่ที่สุด: รายงานข้อมูลที่ถูกเขียนและ address ที่ถูกเขียน

4.2 การทดลองและดีบั๊ก kernel-mode driver

การเขียนโปรแกรม kernel-mode driver จะใช้โปรแกรม Editor ตัวใดก็ได้ในการเขียนแต่ในการ Compile จะใช้ Windows 2000 DDK (Device Driver Development Kit) ในการ compile และการดีบั๊กจะใช้โปรแกรม debugprint ในการดีบั๊ก เพราะไฟล์ที่ได้จากการ compile จะเป็นไฟล์.SYS ดังนั้นวิธีการดีบั๊กจึงมีวิธีการเดียวคือการดู message ที่ถูกส่งออกมาจากโปรแกรม driver ของเรา

ขั้นตอนแรกให้เราทำการ ลง driver ของโปรแกรม debugprint โดยเข้าไปที่ start->Control Panel->Add Hardware จะได้หน้าต่างดังแสดงในรูป 4.2 จากนั้นให้ทำการคลิก Next จะได้หน้าต่างดังแสดงในรูป 4.3

ต่อมาให้เลือก “Yes,I have already connected the hardware” แล้วคลิก Next จะได้หน้าต่างดังแสดงในรูป 4.4

จากนั้นให้เลือก “Add a new hardware device” แล้วคลิก Next จะได้หน้าต่างดังแสดงในรูป 4.5

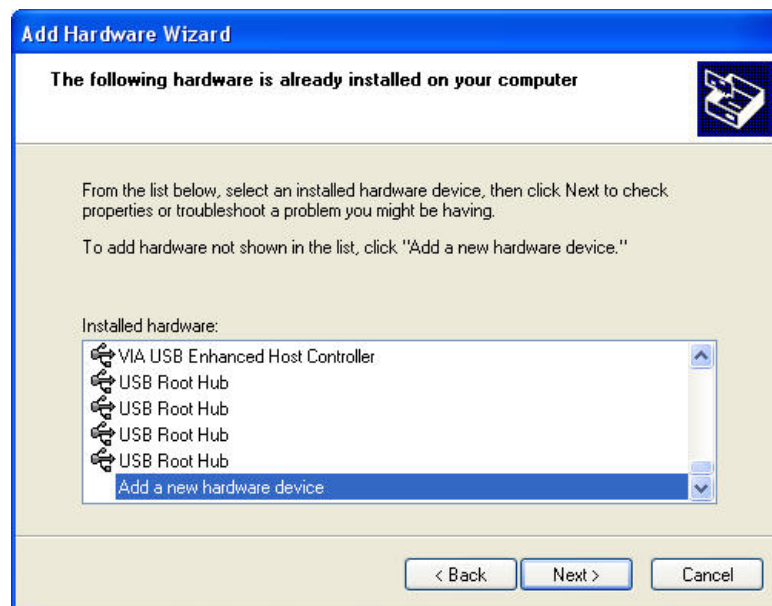
ให้เลือก “Install the hardware that I manually select from a list (Advanced)” แล้วคลิก Next จะได้หน้าต่างดังแสดงในรูป 4.6 เลือก “Show all device” แล้วคลิก Next จะได้หน้าต่างดังแสดงในรูป 4.7 แล้วคลิก “Have Disk” จะได้หน้าต่างดังแสดงในรูป 4.8



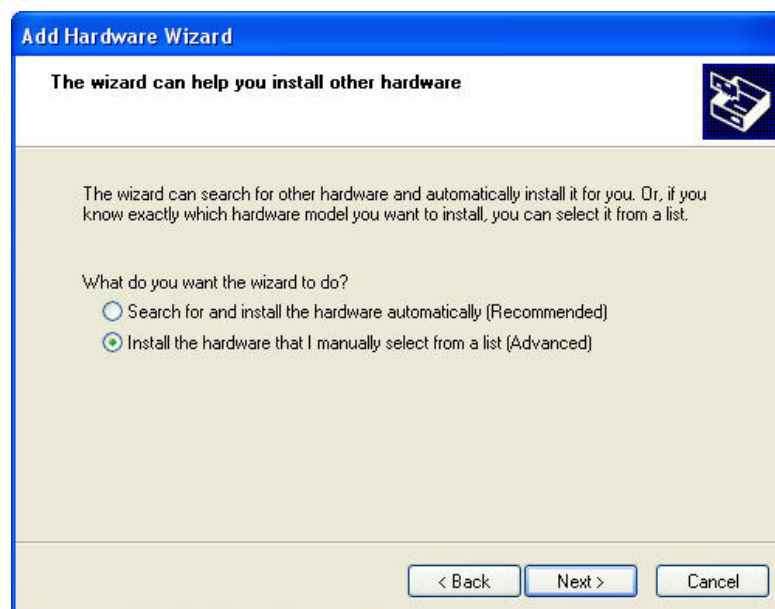
รูปที่ 4.2 การเพิ่ม driver ขั้นตอนที่ 1



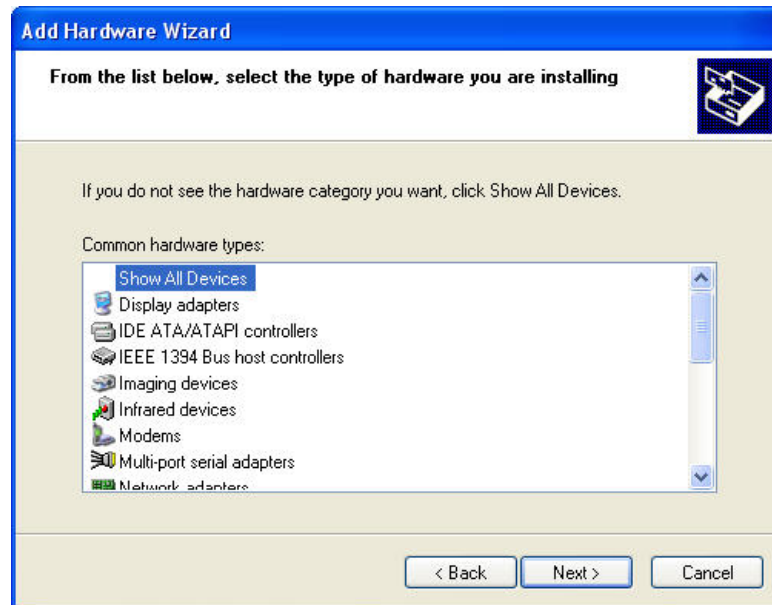
รูปที่ 4.3 การเพิ่ม driver ขั้นตอนที่ 2



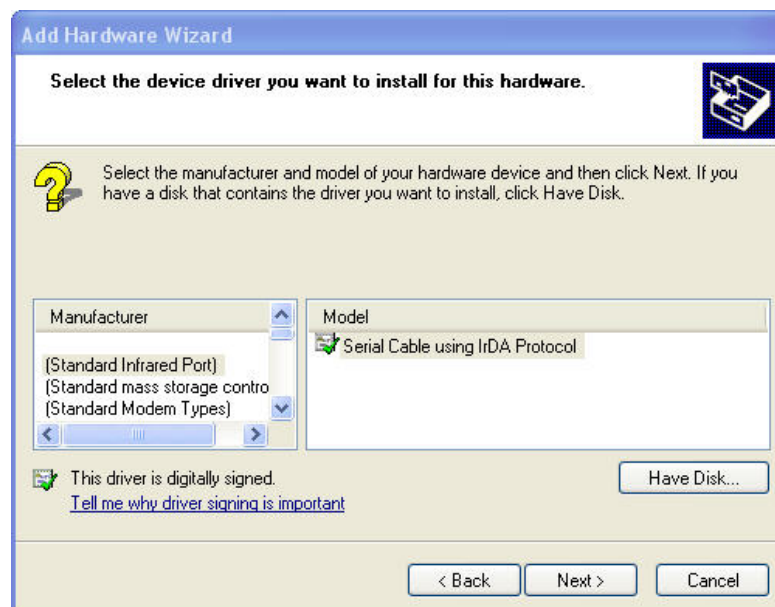
รูปที่ 4.4 การเพิ่ม driver ขั้นตอนที่3



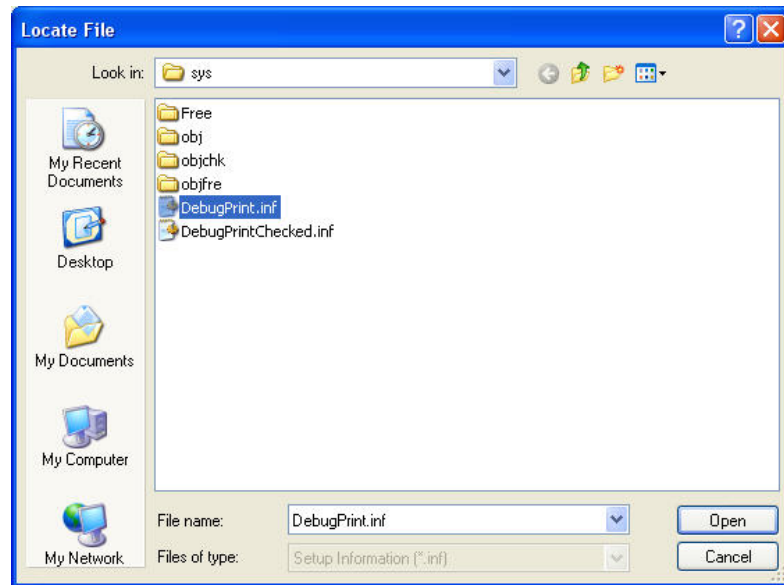
รูปที่ 4.5 การเพิ่ม driver ขั้นตอนที่4



รูปที่ 4.6 การเพิ่ม driver ขั้นตอนที่ 5

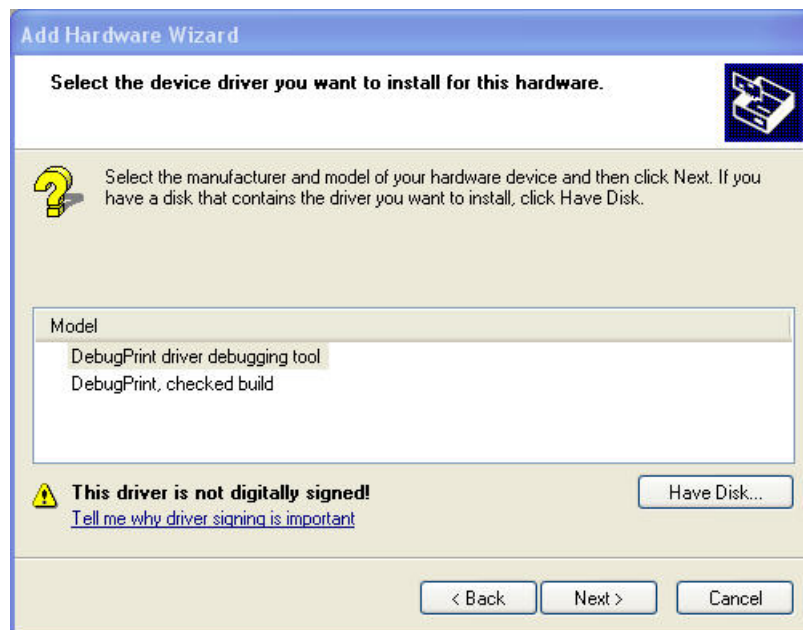


รูปที่ 4.7 การเพิ่ม driver ขั้นตอนที่ 6



รูปที่ 4.8 การเพิ่ม driver ขั้นตอนที่ 7

จากนั้นให้เลือก DebugPrint.inf คลิก Open แล้วคลิก OK จะได้น้ำต่างดังแสดงในรูป 4.8



รูปที่ 4.9 การเพิ่ม driver ขั้นตอนที่ 8

ให้เลือก DebugPrint, checked build คลิก Next แล้วคลิก Next จะเป็นทำการลง driver ของโปรแกรม debugPrint

ต่อมาให้เราทำการเพิ่ม driver ของเราเข้าไปในระบบ โดยเข้าไปที่ start->Control Panel->Add Hardware

จากนั้นเมื่อเราสามารถเพิ่ม driver ของเราเข้าไปในระบบได้แล้ว เราสามารถที่จะตรวจสอบได้ที่ คลิกขวาที่ My Computer->Properties ไปที่แท็บ Hardware คลิกที่ปุ่ม Device manager จะพบกับหน้าต่าง Device Manager ซึ่งจะแสดงรายละเอียดของ driver ที่ได้เพิ่มเข้าไปในระบบแล้ว

ต่อมาให้ทำการเปิดโปรแกรม debugprint ขึ้นมา การจะดู message ที่โปรแกรม debugPrint ส่งออกมานั้นเราจะต้องประกาศการใช้งานไว้ในโค้ดดังนี้

```
DebugPrintInit("String");
```

จะทำการเรียกใช้ฟังก์ชันนี้ภายในฟังก์ชัน DriverEntry เพื่อเป็นการประกาศการใช้งาน โดยค่า String ที่ใส่จะเป็นหัวเรื่องที่ใช้กำหนดค่าให้ debugPrint ส่งข้อความออกมาด้วยหัวเรื่องอะไร และเมื่อมีประกาศการใช้งานเพราะฉะนั้นเมื่อมีการ Uninstall driver ก็จะต้องทำการปิดการส่งข้อความโดยใช้คำสั่งดังนี้

```
DebugPrintClose();
```

ซึ่งจะประกาศเรียกใช้งานที่ฟังก์ชัน DriveUnload จากนั้นถ้าภายในโปรแกรมของ kernel-mode driver ต้องการที่จะส่งข้อความออกไปให้แสดงผลที่โปรแกรม debugPrint นั้นให้ใช้คำสั่งดังนี้

```
DebugPrint("String");
```

โดยที่ string ก็คือข้อความที่เราต้องให้ส่งออกไปแสดงผลที่โปรแกรม debugPrint ประโยชน์ เพื่อดูขณะนี้ driver ของเราทำงานไปถึงที่ฟังก์ชันไหนแล้ว

จากรูปที่ 4.10 จะเป็นการดูข้อความที่ driver ของเราส่งข้อความออกมาที่โปรแกรม debugPrint โดยที่ Column ที่ชื่อ Driver จะแสดงหัวเรื่องที่เราริเรียกใช้จากฟังก์ชัน DriverEntry และจะมีการแสดงเวลาที่ได้รับข้อความที่ driver ส่งเข้ามา และที่ Column Event จะแสดงข้อความที่เราได้ทำการส่งออกมาจากโปรแกรม kernel-mode driver ของเรา จากรูปที่ 4.9 จะเป็นการทดลองการอ่านข้อมูลจากไฟล์.jnk มาเก็บไว้ที่ driver โดยที่ไฟล์.jnk จะเป็นข้อความที่แสดงผลบนคอนส การทดลองนี้ประโยชน์ที่ได้รับคือ รู้ว่าการทำงานของ kernel-mode driver จะมีการทำงานจากฟังก์ชันใดไปยังฟังก์ชันใดบ้าง

Driver	Time	Event
Monitor	12:37:16, 20 Feb 2006	Version 1.03 starting to listen under Windows 2000 (5.1 build 2600) Service Pack 2
Project...	12:37:19	DebugPrint logging started
Project...	12:37:19	DriverEntry Function
Project...	12:37:19	AddDevice Function
Project...	12:37:19	StartDevice Function
Project...	12:37:19	OpenFile Function
Project...	12:37:19	GetFileSize Function
Project...	12:37:19	ReadFile Function
Project...	12:37:19	CloseFile Function
Project...	12:37:47	DispatchCreate Function
Project...	12:37:47	CompleteRequest Function
Project...	12:37:47	DispatchControl Function
Project...	12:37:47	IOCTL_READ_FILE
Project...	12:37:47	CompleteRequest Function
Project...	12:37:47	DispatchClose Function
Project...	12:37:47	CompleteRequest Function
Project...	12:38:00	StopDevice Function
Project...	12:38:00	RemoveDevice Function
Project...	12:38:00	DebugPrint logging ended

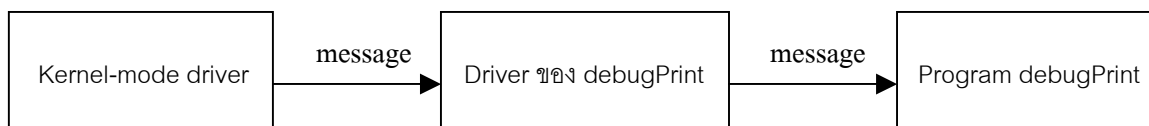
รูปที่ 4.10 การดูข้อความจากโปรแกรม debugPrint

จากรูปที่ 4.10 ในกรอบหมายเลข 1 เป็นขั้นตอนของการกำหนดค่าเริ่มต้นทั้งหมดของ driver โดยการทำงานจะเริ่มจาก ฟังก์ชัน DriverEntry -> AddDevice -> StartDevice -> OpenFile -> GetFileSize -> ReadFile -> CloseFile

ส่วนในกรอบหมายเลข 2 จะเป็นส่วนของการรองรับคำสั่งจาก user-mode driver เมื่อมีการใช้คำสั่ง CreateFile ที่ kernel-mode driver จะมาทำงานที่ฟังก์ชัน DispatchCreate เพื่อทำการส่ง Handle ไปให้ยัง Application จากนั้นเมื่อมีการใช้คำสั่ง DeviceIoControl ที่ kernel-mode driver จะมาทำงานที่ฟังก์ชัน DispatchControl เพื่อรับ IOCTL มาจาก user-mode driver โดยที่ IOCRL ที่รับเข้ามาคือ IOCTL_READ_FILE เพื่อทำการอ่านไฟล์ข้อความที่ driver ได้ทำการอ่านเข้ามาในขั้นตอนแรก แล้วส่งข้อความกลับไปให้ user-mode driver เพื่อส่งข้อความกลับไปให้ Application อีกทีหนึ่ง จากนั้นเมื่ออ่านไฟล์เสร็จที่ user-mode driver ก็จะทำการปิดโปรแกรมโดยใช้คำสั่ง CloseFile การทำงานที่ Kernel-mode driver ก็จะมาทำงานที่ฟังก์ชัน DispatchClose เพื่อทำการปิด Handle ของอุปกรณ์ฮาร์ดแวร์ที่ได้ทำการเปิดไว้

กรอบหมายเลข 3 เป็นการทำงานในส่วนของการ Uninstall Driver เมื่อมีการ Uninstall Driver ก็มีการทำงานที่ฟังก์ชัน Unload Driver เพื่อทำการ deallocate ตัวแปรต่างๆ ที่ได้สร้างขึ้นมาใช้ภายในโปรแกรม

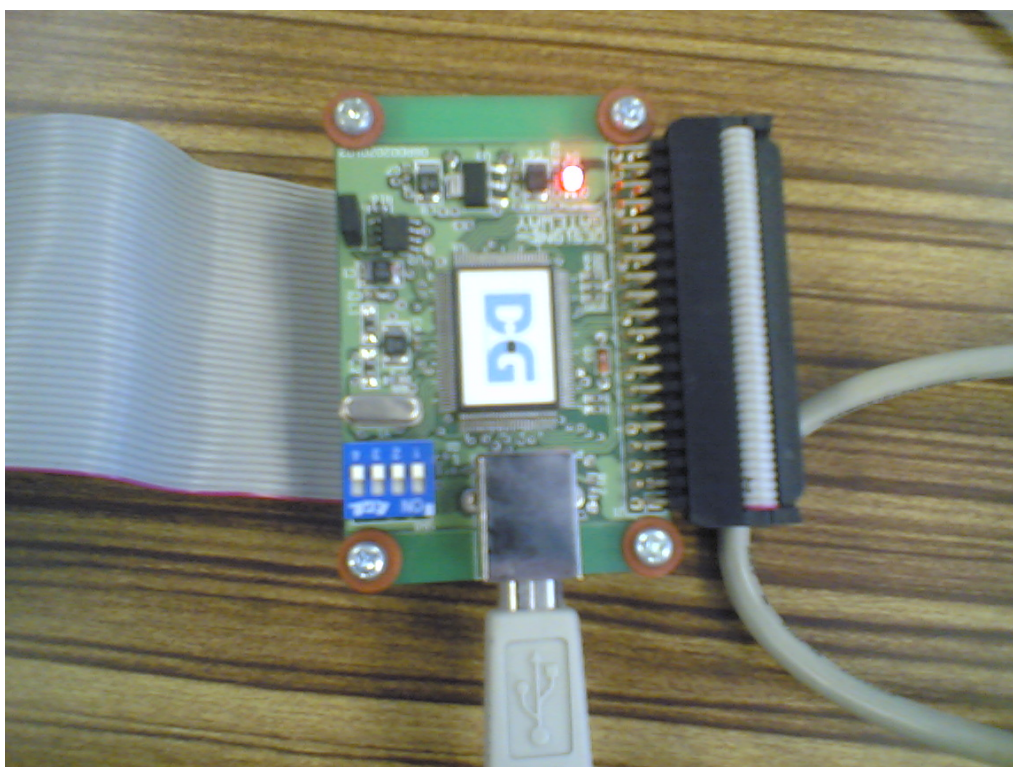
การทำงานระหว่าง kernel-mode driver กับ debugPrint ดังแสดงในรูปที่ 4.10



รูปที่ 4.11 การทำงานระหว่าง kernel-mode driver กับ debugPrint

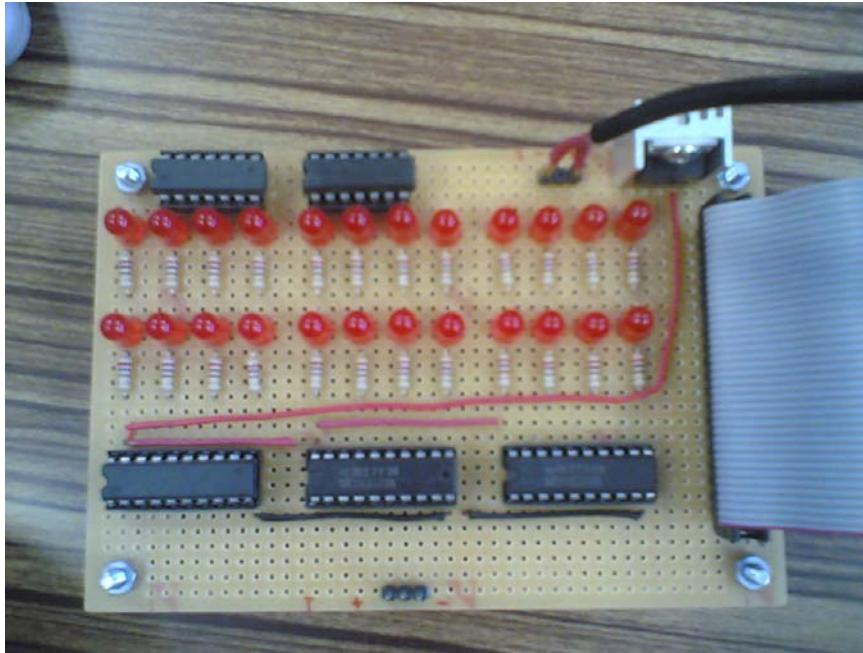
จากรูปที่ 4.11 แสดงการทำงานระหว่าง kernel-mode driver กับ debugPrint โดยที่ kernel-mode driver ต้องการส่งข้อความออกแสดงผลที่โปรแกรม debugPrint ก็จะส่งข้อความไปยัง driver ของโปรแกรม debugPrint เพื่อส่งข้อความไปแสดงผลในโปรแกรม debugPrint

4.3 การทดลองกับอุปกรณ์ฮาร์ดแวร์

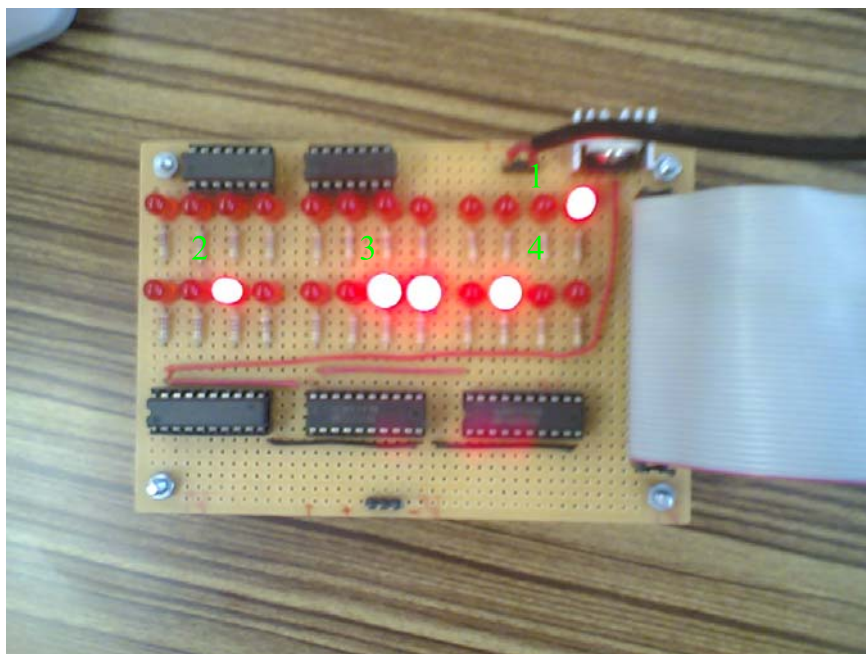


รูปที่ 4.12 แสดงโมดูล EZY I

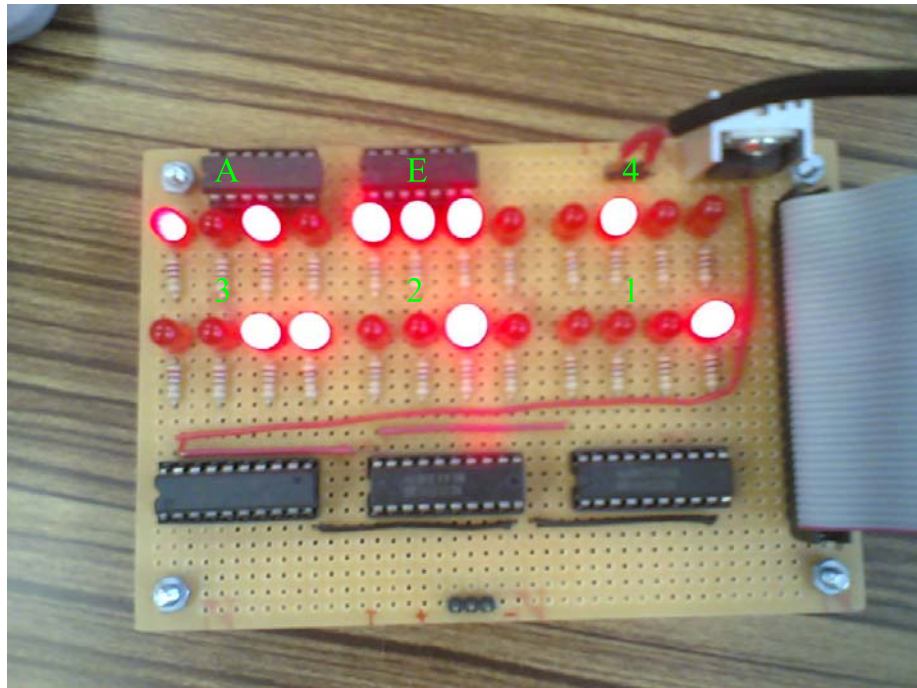
จากรูปที่ 4.12 จะแสดงตัวโมดูลที่นำมาใช้ในการรับส่งข้อมูลผ่านตัว USB ซึ่งจะมีขาสัญญาณอยู่ 40 ขา แบ่งเป็นขาแอดเดรส 8 เส้น และขาข้อมูลอีก 16 เส้น ส่วนที่เหลือก็จะเป็นส่วนของขาที่ใช้ในการควบคุมและขาจ่ายไฟเลี้ยงให้กับโมดูล



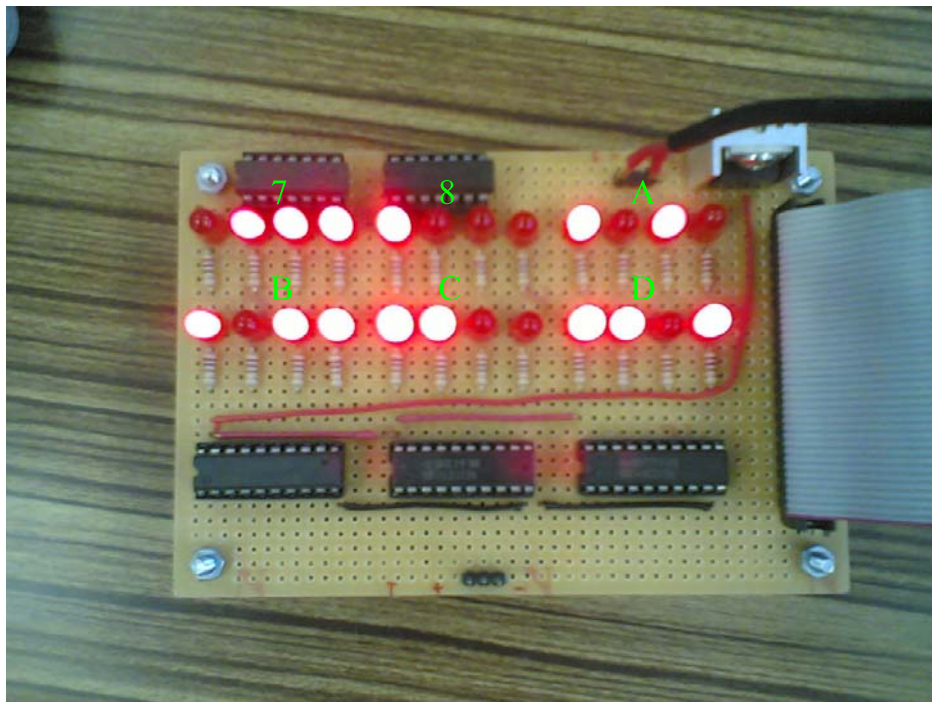
รูปที่ 4.13 แสดงโมดูลที่ใช้การทดลอง



รูปที่ 4.14 แสดงการส่งค่า 0x1234 ที่ address 0x00



รูปที่ 4.15 แสดงการส่งค่า 0x4321 ที่ address 0xAE



รูปที่ 4.16 แสดงการส่งค่า 0xABCD ที่ address 0x78

จากรูปที่ 4.13 จนถึง 4.16 จะเป็นการแสดงการทดสอบป้อนข้อมูลให้กับโมดูล USB เพื่อไปแสดงที่ตัวบอร์ดที่ใช้ในการแสดงผลออกทาง LED ในแถบซ้ายบนสองแถวแรก เราจะใช้เป็น ตัวแสดงแอดเดรสที่เราป้อนเข้าไป ส่วนขวามือบนจะเป็นส่วนแสดงผลของข้อมูลซึ่งจะเป็น high byte และในแถวล่างก็จะเป็นส่วนของข้อมูลซึ่งเรียงจากมากไปน้อยโดยดูจากซ้ายไปขวา อย่างเช่น ในตัวอย่างในรูปที่ 4.16 เราได้ทำการป้อนให้มีแอดเดรส 0x78 และค่าเป็น 0xABCD ซึ่งผลที่ออกมาจะได้ผลลัพธ์อย่างในรูป